

Pflichtenheft

Projekt: Webbasiertes Multiplayer-Spiel 'Schiffe versenken'

Ziel: Technische Beschreibung der Umsetzung und Implementierung (Realisierungskonzept).

1. Systemarchitektur (MVC-Modell)

Zur sauberen Trennung von Logik, Daten und Design wird die Anwendung nach dem etablierten Model-View-Controller (MVC) Architekturmuster strukturiert:

- **Model (Modell):** Die serverseitige Spiellogik in Python. Sie verwaltet den Zustand des Spiels, validiert die Züge und steuert die Datenstrukturen über zweidimensionale Arrays.
- **View (Präsentation):** Die Benutzeroberfläche im Browser. Realisiert mittels HTML5 und CSS3 (Grid-Layout für das 10x10-Spielfeld, Flexbox für Menüs und Layout-Ausrichtungen). JavaScript (ES6) verarbeitet die Klick-Events auf dem Client.
- **Controller (Steuerung):** Die Flask-Routen und Flask-SocketIO-Eventhandler. Sie nehmen Client-Anfragen entgegen, rufen die Logik im Model auf und steuern die Kommunikation zurück an die Views.

2. Technische Umsetzung der Hauptfunktionen

- **Spielfeld-Matrix:** Ein zweidimensionales Array (10x10) im Python-Backend speichert den exakten Zustand jeder Zelle (z. B. 0 = leer, 1 = Schiff, 2 = Fehlschuss, 3 = Treffer).
- **Kommunikations-Protokoll:** Der Datenaustausch erfolgt über WebSockets mittels Flask-SocketIO, um ereignisgesteuerte Updates (z. B. Raumbeitritt, Schussabgabe) ohne Page-Reload zu ermöglichen.
- **Netzwerkinfrastruktur:** Das System läuft lokal auf dem Entwicklungsrechner als Server. Andere Endgeräte verbinden sich über die lokale IP-Adresse des Servers im selben WLAN-Netzwerk.

3. Schnittstellen & WebSocket-Events

- **client_ready:** Wird vom Browser gesendet, sobald der Spieler die Platzierung seiner Schiffe abgeschlossen hat.
- **player_move:** Überträgt die exakten Koordinaten (X/Y) eines Schusses an das Backend.
- **move_result:** Der Server sendet das validierte Ergebnis (Treffer, Wasser oder Versenkt) in Echtzeit an beide Clients zurück.